AI Companion Chatbot

Complete Build Roadmap — 6 Phases · ~4 Months · ■0 to Start

6 Phases	~4 Months	■0 to Start	1 Working App

Phase 01 — Environment + AI brain

Week 1–2

Get Python, APIs, and a talking chatbot working locally

INSTALL	Python 3.11, VS Code, Git, pip, virtualenv	*python.org + code.visualstudio.com*
API SETUP	Get Groq API key (free). Test with a basic chat loop in Python	*console.groq.com*
BUILD	Simple CLI chatbot — type a message, get a reply. No UI yet.	*groq SDK + Python*
PERSONALITY	Write a detailed system prompt: her name, tone, how she responds, memory style	*Plain text in Python*
MEMORY	Store last 20 messages in a list, pass full history to API each call	*Python list / JSON file*

Checkpoint: A working terminal chatbot that remembers context and has a personality

Phase 02 — Voice — she speaks and listens

Week 3–4

Add microphone input and natural voice output

SPEECH-TO-TEXT	Install openai-whisper locally. Record mic, transcribe, send to LLM	*pip install openai-whisper + pyaudio*
TEXT-TO-SPEECH	Install edge-tts (free Microsoft voices). Generate audio from reply text	*pip install edge-tts*
PLAYBACK	Auto-play the audio reply using pygame or playsound	*pip install pygame*
LOOP	Make it continuous: listen, transcribe, reply, speak, listen again	*Python while loop + threading*
VOICE PICK	Browse edge-tts voices, find one that fits. en-IN or en-US feminine voices	*edge-tts --list-voices*

Checkpoint: Full voice conversation loop — you speak, she replies in her voice, fully offline

Phase 03 — The face — 2D talking avatar

Week 5–7

A girl on screen who lip-syncs to her voice (easiest path first)

PICK AN IMAGE	Download a free anime-style girl PNG. This is her face.	*Stable Diffusion locally or itch.io free assets*
LIP-SYNC ENGINE	Install SadTalker (open source). Feed: her image + audio file, outputs a talking video	*github.com/OpenTalker/SadTalker*

PIPELINE	After LLM replies, generate audio via edge-tts, feed into SadTalker, display video	*Python subprocess + cv2*
DISPLAY	Show output video in a Tkinter or PyQt window fullscreen-style	*pip install PyQt5 + cv2*
LOOP VIDEO	When not speaking, loop an idle animation. Use a pre-made idle clip.	*ffmpeg + cv2*

Checkpoint: A window shows a girl who visibly talks when she replies — real lip movement from audio

Phase 04 — Web UI — the video call interface

Week 8–10

Move everything into a browser-based app that looks like a real video call

BACKEND API	Build FastAPI server. Endpoints: POST /chat (text in, text out), POST /speak (text, audio)	*pip install fastapi uvicorn*
FRONTEND	React app with: avatar video panel, chat log, mic button, speaker. Video-call layout.	*npx create-react-app or Vite*
AUDIO STREAM	Browser records mic via MediaRecorder API, sends to /transcribe, gets reply, plays audio	*Web Audio API + fetch*
AVATAR PANEL	Video element plays idle.mp4 by default. On reply, swap to talking.mp4 from SadTalker	*HTML5 video element*
POLISH UI	Dark background, glowing avatar border when speaking, subtle chat overlay at bottom	*Tailwind CSS*

Checkpoint: A localhost web app that looks and feels like a video call with your AI companion

Phase 05 — Train it on your data

Week 11–13

Feed it your personality, memories, and style so it truly knows you

COLLECT DATA	Export WhatsApp chats, notes, diary entries, old texts. Clean into plain text.	*WhatsApp export + Python script to clean*
RAG SETUP	Chunk your data into 200-word blocks. Embed with sentence-transformers. Store in ChromaDB	*pip install chromadb sentence-transformers*
MEMORY INJECT	On each message, search ChromaDB for relevant memories, inject top 3 into system prompt	*Python + ChromaDB query*
STYLE FINE-TUNE	Optional: use Unsloth to LoRA fine-tune Llama 3 on your chat data on free Colab GPU	*Google Colab free T4 + Unsloth*
PERSONA LOCK	After each session, summarize and save key facts to a JSON file she recalls next time	*LLM summarization + JSON file*

Checkpoint: She remembers things you told her, knows your habits, and speaks in a style trained on your data

Phase 06 — Deploy + run 24/7

Week 14–16

Make it accessible from your phone and keep it running always

LOCAL SERVER	Run FastAPI on your PC 24/7. Use Ngrok free tunnel to expose it to the internet securely	*ngrok free tier + uvicorn*
OR CLOUD	Deploy backend to Render.com free tier or Railway. SadTalker needs GPU — use Replicate	*render.com or railway.app*
MOBILE ACCESS	Your React app works on mobile browser. Add to homescreen as PWA for app-like experience	*manifest.json + service worker*
AUTH	Add a simple password gate so only you can access it. JWT token or HTTP basic auth.	*FastAPI security utils*
UPGRADES	Swap edge-tts for ElevenLabs when budget allows. Upgrade avatar to 3D VRM + Three.js later.	*ElevenLabs free tier to start*

Checkpoint: Accessible from your phone anywhere, running 24/7, fully yours and private

---

MacBook Air M3 8GB 256GB — Compatibility Notes

Component	Status	Notes
Groq API (LLM)	**Fully supported**	Cloud API, no local GPU needed. Works perfectly.
Whisper (speech-to-text)	**Fully supported**	Runs great on M3. Use whisper 'tiny' or 'base' model for speed.
edge-tts (text-to-speech)	**Fully supported**	Pure Python, no GPU needed. Works on all platforms.
SadTalker (lip-sync)	**Works with limits**	No CUDA on Mac. Runs on CPU — slow (~30–60s per video clip). Use short clips.
FastAPI backend	**Fully supported**	Lightweight Python server. M3 handles it easily.
React frontend	**Fully supported**	Node.js runs natively on M3. No issues.
ChromaDB (RAG memory)	**Fully supported**	Runs locally, no GPU needed. Very fast on M3.
Llama 3 via Ollama	**Fully supported**	M3 handles 7B models well with 8GB RAM. Use llama3:8b.
LoRA fine-tuning (Unsloth)	**Not recommended**	Use Google Colab free GPU instead. 8GB RAM is tight for training.
256GB storage	**Tight but fine**	Llama 3 8B model = ~5GB. SadTalker weights = ~3GB. Plan your space.

Verdict: Your MacBook Air M3 handles 90% of this roadmap perfectly. The only bottleneck is SadTalker lip-sync — it runs on CPU so expect 30–60 seconds per video clip. For fine-tuning (Phase 5 optional step), use Google Colab's free T4 GPU instead of your Mac. The 8GB RAM is sufficient — Llama 3 8B via Ollama fits comfortably. Watch your 256GB storage as model weights add up.